Part 7: Feed-Forward Network, SwiGLU, and the First Predicted Token

These are companion notes for the Part 7 video. They develop the feed-forward section of my working notes into a code walkthrough. Part 6 finished self-attention; this part finishes each Llama layer, runs all 16 layers, and selects one next token. Part 8 later adds cached, token-at-a-time generation.

The code excerpts below use the **prompt prefill that predicts one next token**, shown in the video and committed as `e9399f9`. Some excerpts show the calls without their surrounding error checks so the data flow is easier to follow. The current `src/main.cpp` also has the later KV-cache, decode, batching, and paging paths. To see the complete video-era source rather than the later additions, run `git show e9399f9:src/main.cpp` and `git show e9399f9:src/kernels.cu`.

Where this work fits

For one token position, let `h` be the hidden state entering a decoder layer. Attention has already mixed in information from the allowed earlier positions:

```
a = h + Attention(RMSNorm(h))
x = RMSNorm(a)                          // second, post-attention norm
gate = x × W_gateᵀ                      // 2048 -> 8192
up   = x × W_upᵀ                        // 2048 -> 8192
m    = SiLU(gate) ⊙ up                  // elementwise, still 8192
down = m × W_downᵀ                      // 8192 -> 2048
next_hidden = a + down                  // second residual
```

The same sequence runs for every token position and then repeats with **different weights** in layers 0 through 15. After layer 15, the last prompt position goes through a final RMSNorm, the language-model head, and argmax. This video ends after predicting **one** next token; it does not yet loop to generate a response.

Why have a feed-forward network after attention?

An analogy from my rough notes: imagine each token as a person holding a page of numbers. Attention is the round in which people can exchange information with permitted earlier people. The feed-forward network, or **FFN**, is the next round: each person applies the same kind of learned processing to *their own updated page*. The FFN does not perform another exchange between token positions.

That does **not** mean the FFN's input has no context. Attention has already put context from earlier tokens into each updated hidden vector. The FFN processes those context-aware features one position at a time. In code, using one matrix multiplication for all prompt rows is a faster way to apply the same layer's weights independently to every row; it does not mix rows together.

The weights were learned during training. During this inference run we read them and calculate outputs; we are not adjusting them to make the answer better. A different token vector produces different gate and up values even though this layer's weights stay fixed.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Build the idea from one perceptron

A simple perceptron combines inputs with weights, adds a bias, and applies a decision rule. Suppose an imaginary urgency detector gets `x[0] = 1` when a message contains “urgent” and `x[1]` equal to its exclamation-mark count:

```
float score = 1.5f * x[0] + 0.25f * x[1] - 1.0f;
bool is_urgent = score > 0.0f;
```

For `x = [1, 2]`, the score is `1.5 + 0.5 - 1 = 1`, so the toy detector says yes. The weights decide how much each input contributes; the `-1` is a **bias** that moves the decision boundary. These values are hand-chosen for the example. Real model weights are learned.

Remove the yes/no threshold for a moment and imagine thousands of different weighted sums of the **same** input. Put each sum's weights in one row of a matrix. A matrix multiplication calculates all those sums and gives us a new vector. This is a **linear projection**. The projections in this Llama MLP produce numerical features, not urgent/not-urgent booleans, and they have **no bias**.

A multilayer perceptron, or **MLP**, puts learned projections and nonlinear operations together. Nonlinearity matters: if we only applied one linear matrix and then another, their product could be replaced with one matrix. SiLU and the multiplication of two branches let the MLP represent relationships that a single linear projection cannot.

Llama's MLP has a particular gated shape. The gate and up projections are **parallel**: both read the same normalized input `x`. We do not feed the up output into the gate projection or the gate output into the up projection. Only after both exist do we combine their values.

Follow the shapes before the code

Let `T` be the number of prompt tokens. A weight matrix is listed as `[output features, input features]` because each output feature owns one row of weights.

Item	Shape	Purpose
`activations_gpu` after attention residual	`[T, 2048]`	Hidden state to preserve for the MLP skip connection.
`normalized_gpu`	`[T, 2048]`	Post-attention RMSNorm output supplied to both branches.
`W_gate`, `W_up`	`[8192, 2048]` each	Two different learned views of the same input.
`gate_gpu`, `up_gpu`	`[T, 8192]` each	One gate/up value per token and intermediate feature.
`gate_gpu` after SwiGLU	`[T, 8192]`	`SiLU(gate) * up`, written over the old gate values.
`W_down`	`[2048, 8192]`	Learned mixing back to hidden width.
`output_projection_gpu` after down projection	`[T, 2048]`	MLP update to add to `activations_gpu`.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

For a **single** token, the up projection is `[1, 2048] × [2048, 8192] = [1, 8192]`. Each of the 8,192 answers is a dot product between the 2,048-value input and one row of `W_up`. The gate projection repeats that shape with its own weights. “Up” means more intermediate features, not copying values or padding with zeros. “Down” uses learned weights to mix the 8,192 values back to 2,048; it does not simply discard the extra positions.

After the MLP residual, every token still has a 2,048-value hidden vector. That unchanged outer shape is why the result can be passed into the next decoder layer.

What SiLU and the gate actually do

For one gate value `g`:

```
sigmoid(g) = 1 / (1 + e^(-g))
SiLU(g)    = g * sigmoid(g) = g / (1 + e^(-g))
output[j]  = SiLU(gate[j]) * up[j]
```

`sigmoid(g)` lies between zero and one, but **SiLU(g) need not**. It may be negative or greater than one. So “gate” is an analogy, not a strict binary switch. For the small examples in my working notes:

```
gate                = [ 2,      -2,       0]
up                  = [ 3,       4,       5]
SiLU(gate)          ≈ [ 1.7616, -0.2384, 0]
SiLU(gate) ⊙ up     ≈ [ 5.2848, -0.9536, 0]
```

The first up value is amplified, the second changes sign and becomes smaller in magnitude, and the third contributes zero. The real vector has 8,192 entries. ⊙ means **elementwise multiplication**: entry 0 times entry 0, entry 1 times entry 1, and so on. It is not a dot product, so the result remains 8,192 values. This SiLU-and-multiply arrangement is the **SwiGLU** step in this implementation.

SiLU is smooth, which helped make these weights trainable with gradients. The inference code only evaluates the learned function; it does not calculate gradients or update weights.

Code: locate each layer's weights

The loader keeps direct GPU pointers for every layer's MLP matrices. The following assignments appear in the model loader in `src/main.cpp`:

```
weights.mlp_gate_proj[layer] = reinterpret_cast<__nv_bfloat16 *>(
    base + offsets.at(prefix + ".mlp.gate_proj.weight"));
weights.mlp_up_proj[layer] = reinterpret_cast<__nv_bfloat16 *>(
    base + offsets.at(prefix + ".mlp.up_proj.weight"));
weights.mlp_down_proj[layer] = reinterpret_cast<__nv_bfloat16 *>(
    base + offsets.at(prefix + ".mlp.down_proj.weight"));
```

`base` points at the model data already copied to GPU memory. `offsets.at(...)` finds the byte offset recorded in the safetensors header. `reinterpret_cast` makes a typed BF16 pointer at that location; these lines do not run the MLP or copy another matrix. The layer number becomes part of `prefix`, so layer 0 and layer 1 point at different learned tensors. Within one layer, all prompt positions use that same layer's tensors.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Code: normalize, then compute both branches

In video-era `prefill`, `gate_gpu` and `up_gpu` are allocated as `T × 8192` BF16 elements **before** the 16-layer loop and reused in each layer. After the attention projection has been added to `activations_gpu`, the MLP path starts:

```
launchRmsNorm(
    activations_gpu,
    normalized_gpu,
    weights.post_attn_layernorms[layer],
    token_count);

const cublasStatus_t gate_status = projectBf16RowMajor(
    cublas_handle, normalized_gpu, weights.mlp_gate_proj[layer],
    gate_gpu, token_count, HIDDEN_SIZE, INTERMEDIATE_SIZE);

const cublasStatus_t up_status = projectBf16RowMajor(
    cublas_handle, normalized_gpu, weights.mlp_up_proj[layer],
    up_gpu, token_count, HIDDEN_SIZE, INTERMEDIATE_SIZE);
```

`launchRmsNorm` reads `activations_gpu` and writes a separate `normalized_gpu`. We covered the RMSNorm calculation in Part 3; the important detail here is **which buffer survives**. `activations_gpu` retains the hidden state from after the attention residual, ready for the second residual later. Both projection calls read `normalized_gpu`. Their different weight pointers and different output buffers make them parallel branches, although these host calls are issued in sequence.

`projectBf16RowMajor` is the same cuBLAS helper used for Q/K/V. Its last three arguments are the token count, input width, and output width. Here they are `T`, `2048`, and `8192`. Conceptually it computes `input × weightᵀ` in row-major layout. The central cuBLAS arguments are:

```
CUBLAS_OP_T, CUBLAS_OP_N,
output_features, token_count, input_features,
&alpha, weight, CUDA_R_16BF, input_features,
input, CUDA_R_16BF, input_features,
&beta, output, CUDA_R_16BF, output_features,
CUBLAS_COMPUTE_32F, CUBLAS_GEMM_DEFAULT
```

The output width is cuBLAS's `m`, the token count is `n`, and the input width is the shared reduction size `k`. `CUBLAS_OP_T` on the weight and `CUBLAS_OP_N` on the input account for cuBLAS interpreting the same row-major bytes as column-major matrices; no GPU buffer is physically transposed. `alpha = 1` applies the product, and `beta = 0` overwrites old scratch contents. The multiplication accumulates with `CUBLAS_COMPUTE_32F`, then stores these branch outputs as BF16. The two `cublasStatus_t` results are checked so a failed projection does not silently continue.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Code: one CUDA thread per SwiGLU value

`swiGluKernel` implements the formula literally:

```
const size_t index = static_cast<size_t>(blockIdx.x) * blockDim.x + threadIdx.x;
if (index >= value_count)
{
    return;
}

const float g = __bfloat162float(gate[index]);
const float u = __bfloat162float(up[index]);
gate[index] = __float2bfloat16((g / (1.0f + expf(-g))) * u);
```

`blockIdx.x * blockDim.x + threadIdx.x` gives one flat index across all `T × 8192` values. The bounds check protects the last CUDA block when the count is not a multiple of 256. The two BF16 inputs become `float` for the exponential and multiply. `g / (1 + expf(-g))` is SiLU. Multiplying by `u` produces the gated value, then `__float2bfloat16` rounds it back to BF16.

The write goes into `gate[index]` **in place**. After this kernel, `gate_gpu` no longer contains raw gate scores; it contains the MLP intermediate `SiLU(gate) ⊙ up`. The up buffer is read but not modified. The launcher uses 256 threads per block and enough blocks to cover all `T × 8192` entries. This is an elementwise kernel: a thread needs no values from another token or another intermediate feature.

Code: project down and add the correct residual

The host then uses the modified `gate_gpu` as the down projection input:

```
launchSwiGlu(gate_gpu, up_gpu, token_count, INTERMEDIATE_SIZE);
projectBf16RowMajor(
    cublas_handle,
    gate_gpu,
    weights.mlp_down_proj[layer],
    output_projection_gpu,
    token_count,
    INTERMEDIATE_SIZE,
    HIDDEN_SIZE);
launchResidualAdd(
    activations_gpu,
    output_projection_gpu,
    token_count,
    HIDDEN_SIZE);
```

The down projection multiplies `[T, 8192]` by the transpose of `[2048, 8192]` and writes `[T, 2048]`. `output_projection_gpu` held the attention projection earlier, but that value has already been added to `activations_gpu`, so its allocation can now hold the MLP projection. Reusing the buffer avoids another `T × 2048` allocation.

`launchResidualAdd` modifies `activations_gpu` in place: `activations_gpu += output_projection_gpu`. Its left side is the hidden state **from before the second RMSNorm**, after the attention residual. Adding the MLP result to `normalized_gpu` would change the model. The code groups these calls under error checks and synchronizes afterward to catch a GPU execution failure before starting the next layer.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Code: repeat the whole decoder layer 16 times

The video-era prefill wraps the original attention path and the new MLP path in:

```
for (int layer = 0; layer < N_LAYERS; ++layer)
{
    // Input norm -> Q/K/V -> RoPE -> causal GQA -> output projection.
    // Attention residual -> post-attention norm -> gate/up -> SwiGLU.
    // Down projection -> MLP residual.
}
```

`N_LAYERS` is 16. This is one complete forward pass through the same **sequence of operations**, not 16 copies of the same learned weights. The arrays `weights.w_q[layer]`, `weights.post_attn_layernorms[layer]`, `weights.mlp_gate_proj[layer]`, and the other layer-indexed pointers select that layer's tensors. The same scratch buffers can be reused because only the completed `activations_gpu` must pass to the next layer. Its shape stays `[T, 2048]`, while its numerical contents change at each layer. We do not select a token after each layer.

Code: from the final hidden vector to vocabulary scores

After layer 15, only the **last prompt position** is needed to predict what follows the prompt. Causal attention lets that position incorporate itself and all earlier prompt positions. The video-era code selects its 2,048-value row and applies the final RMSNorm:

```
const __nv_bfloat16 *last_hidden =
    activations_gpu + static_cast<size_t>(token_count - 1) * HIDDEN_SIZE;
launchRmsNorm(last_hidden, normalized_gpu, weights.norm, 1);
```

The offset skips `token_count - 1` complete hidden rows. Passing `1` normalizes just that row, so we do not spend work projecting every prompt position into vocabulary scores when we only need the first response token.

The **LM head** (“language-model head”) is a matrix with logical shape `[128256, 2048]`: one 2,048-weight row per vocabulary token. Its dot product with the normalized last hidden row produces **128,256 logits**, one score for each possible next token:

```
logit[token_id] = sum over feature=0..2047 of
                  normalized_last_hidden[feature] * LM_head[token_id, feature]
```

The loader uses `lm_head.weight` if the safetensors file provides it; otherwise it points `weights.lm_head` at `weights.embed_tokens`. That fallback is **weight tying**: the same learned table serves both input embeddings and output scoring. It is still an output projection, but we should not assume every checkpoint stores a separate LM-head tensor.

The code calls cuBLAS with `VOCAB_SIZE, 1, HIDDEN_SIZE` as its output rows, token columns, and reduction width:

```
cublasGemmEx(
    cublas_handle, CUBLAS_OP_T, CUBLAS_OP_N,
    VOCAB_SIZE, 1, HIDDEN_SIZE,
    &alpha, weights.lm_head, CUDA_R_16BF, HIDDEN_SIZE,
    normalized_gpu, CUDA_R_16BF, HIDDEN_SIZE,
    &beta, logits_gpu, CUDA_R_32F, VOCAB_SIZE,
    CUBLAS_COMPUTE_32F, CUBLAS_GEMM_DEFAULT);
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

The input and weights are BF16, the GEMM computes in FP32, and `logits_gpu` stores FP32. These are **scores**, not probabilities: a logit of `5` does not mean a 50% chance. Higher logits rank ahead of lower logits for this one prediction.

Code: argmax returns a token ID, not its score

My rough-note example uses a five-token vocabulary:

```
token ID:    0     1     2     3     4
logit:      0.4  -1.1   3.7   2.0  -0.3
```

The largest **value** is `3.7`; its **index** is token ID `2`. Argmax returns `2`. For the real 128,256-entry logit vector, `argmaxKernel` launches one block of 256 threads. Each thread scans indices `threadIdx.x`, `threadIdx.x + 256`, and so on, keeping its best score and ID. The kernel then reduces the 256 candidates through shared memory:

```
for (int index = thread; index < vocabulary_size; index += THREADS)
{
    const float value = logits[index];
    if (value > best_value || (value == best_value && index < best_index))
    {
        best_value = value;
        best_index = index;
    }
}
best_values[thread] = best_value;
best_indices[thread] = best_index;
__syncthreads();
```

At this vocabulary size, each thread examines 501 entries. The shared-memory reduction then halves the candidate count on every pass:

```
for (int stride = THREADS / 2; stride > 0; stride >>= 1)
{
    if (thread < stride)
    {
        const float other_value = best_values[thread + stride];
        const int other_index = best_indices[thread + stride];
        if (other_value > best_values[thread] ||
            (other_value == best_values[thread] && other_index < best_indices[thread]))
        {
            best_values[thread] = other_value;
            best_indices[thread] = other_index;
        }
    }
    __syncthreads();
}
```

The comparison chooses the **smaller token ID** when scores tie, so argmax is deterministic. Thread 0 writes the winning ID to `next_token_gpu`; `cudaMemcpy` brings the single integer back to the CPU. In the video-era `main`, the final print is:

```
std::cout << "Next token ID: " << next_token_id << '\n';
std::cout << "Next token: " << tokenizer.decode({next_token_id}) << '\n';
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

`tokenizer.decode` turns the ID into its text or bytes. One token might be a whole word, part of a word, punctuation, or another token piece.

This is **greedy** selection. We do not need a vocabulary softmax first: exponentiation and division by the same positive sum preserve the ordering of finite logits, so the largest logit would also have the largest softmax probability. Attention's softmax has a different job: it produces weights for mixing value vectors.

Keep these distinctions straight

- The attention output projection `W_o` is a **weight matrix**. Its computed output, after adding the attention residual, is the hidden state that enters the second norm.
- The FFN does no *new* cross-token mixing. Its input can already contain context gathered by attention.
- Gate and up are two projections from the **same** normalized hidden vector. Their outputs are multiplied elementwise after SiLU.
- `sigmoid(g)` is between zero and one; the full `SiLU(g) * up` result can amplify a value, change its sign, or make it zero.
- The MLP residual adds to the hidden state **before** the post-attention RMSNorm, not to that norm's output.
- A complete layer preserves hidden width 2,048. All 16 layers use their **own** learned weights.
- The LM head produces logits. Argmax chooses the **token ID** at the highest logit; the tokenizer turns that ID into readable output.
- The Part 7 code predicts one token from prefill. Part 8 explains how subsequent predicted tokens are fed back through the model.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.